Compare Three Classifiers' Robustness on Intel Image Classification Dataset

Guangheng (Jack) Tong (1129548) Khue Nguyen (1263494) Jake Gerstel (1579814)

1. Introduction

We studied scene recognition on the Intel Image Classification dataset (six classes: buildings, forest, glacier, mountain, sea, street). It has around 25k images of size 150×150, including 3 splits (seg_train, seg_test, seg_pred).

Source URL: https://www.kaggle.com/datasets/puneet6060/intel-image-classification/data.

Research question: Which approach yields better accuracy, data efficiency, and robustness to common perturbations, including occlusion, lighting, and view angles on the Intel Image Classification dataset: classic feature-based pipelines (GaussianNB, RBF-SVM) or a modern CNN (EfficientNet)?

2. Literature Review

Dataset Reference: The dataset was initially published on https://datahack.analyticsvidhya.com by Intel to host an image classification challenge.

Support Vector Machine (Boser et al., 1992) introduces maxim-margin training and the dual formation that enables kernel methods (e.g. Radial Basis Function) with capacity control via support vectors. This justifies our RBF-SVM baseline.

EfficientNet (Tan & Le, 2019) proposes compound scaling of depth, width, and resolution with a single coefficient, delivering better accuracy parameter trade-offs than prior CNNs. It is built on mobile inverted bottleneck convolution blocks with squeeze-and-excitation. In our setup, we fine-tune EfficientNet-B0 with nested cross validation to select learning rate.

3. Methods

Data Curation: We disregard unlabelled seg_pred and merged labelled seg_train and seg_test into a single corpus, and train on a class-balanced subset (1,667 images per class, randomly selected) to control sample size while enabling fair comparison. Per-color channel mean/std (means = [0.4315, 0.4574, 0.4528], stds = [0.2179, 0.2187, 0.2567]) were computed over the subset.

Preprocessing Pipelines:

Train pipeline (used for all models):

1. Resize to (10, 10) with bilinear interpolation. (We chose 10×10 due to computation limitation.)
2. RandomRotation (±15°): pose invariance.
3. ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2): illumination/contrast invariance.
4. GaussianBlur(kernel_size=3, $\sigma \in [0.5,1.0]$): low-pass robustness.
5. ToTensor: scales to [0,1].
6. AddGaussianNoise(σ=0.05): sensor noise robustness.
7. Normalize(mean, std): per-channel standardization with stats above.
8. RandomErasing(p=0.5, area 10–20%, ratio 0.5–2.0): occlusion invariance.

Rationale: Light geometric + photometric jitter combats overfitting; Noise + blur improve stability; Erasing simulates occlusion; Normalization stabilizes optimization and, for PCA/NB/SVM, centres features.

Validation pipeline (clean, used for validation only): Resize → ToTensor → Normalize.

Robustness Test Pipelines:

- Occlusion: Resize → ToTensor → Normalize → RandomErasing(p=1.0, area 20–40%, ratio 0.5–2.0).
- Lighting: Resize → ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4) → ToTensor → Normalize.
- View: Resize → RandomRotation($\pm 25^\circ$) → RandomPerspective(distortion=0.1, p=1.0) → ToTensor → Normalize.

Algorithms: Three algorithms were implemented using scikit-learn and PyTorch, selected for their complementary strengths.

- Gaussian Naïve Bayes (GNB) (High-bias/Low-variance): Fast generative baseline with small feature dimension. PCA compresses raw pixels to decorrelated components that better match GNB's conditional independence assumption and avoids singular covariance. However, it misses interacting feature structures and thus, low in accuracy.
- RBF-SVM (Medium-bias/Medium-variance): PCA reduces noise and computation; Radial basis function (RBF) kernel provides strong non-linear decision boundaries on compact feature spaces; Scaling (gamma='scale') avoids feature-scale bias. It can handle complex class shapes, but it has a high computational cost on large dataset.

- EfficientNet (Low-bias/high-variance): Modern CNN family (B0) use mobile inverted bottleneck convolution blocks and squeeze-and-excitation. It is a strong transfer from ImageNet and has high representational power. However, it is heavier to compute and could risk overfitting without regularization.

Cross-Validation: A nested cross-validation framework ensured robust model evaluation.

- Outer Cross-Validation: 10-fold cross-validation divided the dataset (all 10,002 instances) into 10 equal parts.
- Inner Cross-Validation: 3-fold cross-validation within each training set tuned hyperparameters.
- Process: For each outer fold, the inner cross-validation tested hyperparameter combinations, selecting the best via macro-F1 score. The outer loop then evaluated this model on the held-out test fold. This nested structure avoids information leakage, as test data remains unseen during tuning.
    - Metrics (Accuracy, Precision, Recall, marco-F1 score, Confusion Matrix) are computed per fold, averaged across outer folds for final scores.
    - Learning curves: for each outer fold and its selected configuration, we train on stratified fractions of the outer-train (10%, 20%, ..., 100%) and evaluate on training, evaluation, and evaluation under robustness transfer (occlusion/lighting/view) F1 score. We then plot mean ± std across outer folds.

Hyperparameter Tuning: Grid search within the inner cross-validation to optimize hyperparameters.

- Naïve Byes: Tune PCA $\in$ {16, 32, 64}, var_smoothing $\in$ {1e-4, 1e-3, 1e-2}
    - PCA components (feature count after PCA): Reduce noise and correlations so Gaussian NB's Gaussian and independent feature assumption is less violated. It also keeps the dimension small to avoid variance spikes.
        - Higher PCA: lower bias, higher variance, more computation; It may risk overfitting.
        - Lower PCA: higher bias, lower variance, less computation; It may discard discriminative variance.
    - Var_smoothing (ε added to variances): Prevents zero/near-zero variance and numerical instability.
        - Higher ε: More smoothing, safer numeric, higher bias, flatter posteriors.
        - Lower ε: Sharper likelihoods; lower bias; more sensitive to noise and outliers.
- RBF-SVM: Tune PCA $\in$ {16, 32, 64}, C $\in$ {0.1, 1, 10}
    - PCA components: De-noise and control runtime; RBF distance is sensitive to noisy axes.
        - Higher PCA: capture finer structure, but may inflate variance and kernel sensitivity.
        - Lower PCA: Enforces smoother boundaries, but risks underfitting if too low.
    - C (misclassification penalty, regularization strength): Trade-off margin vs training errors.
        - Higher C: Tighter fit, smaller margin, lower bias, higher variance, can overfit.
        - Lower C: Wider margin, higher bias, lower variance, more robust but may underfit.
- EfficientNet: Tuned LR $\in$ {1e-3, 5e-3, 1e-2}, WD = 0.3 (fixed epochs = 10 due to computation limit)
    - Learning rate (LR): higher means faster search but can overshoot; lower means slower search but safer.
        - Higher: more shrinkage, underfitting.
        - Lower: less shrinkage, better fit but risk overfitting.
    - Weight decay (WD): L2-style parameter shrinkage (decoupled in AdamW optimizer)
        - We fixed WD = 0.3 after tuning WD $\in$ {1e-3, 1e-2, 1e-1, 3e-1, 5e-1} on a random split. It offers strong regularization and reduces variance. (This is not included in Grid search due to computation limit.)

The best configurations were selected by maximizing inner cross-validation F1 score, enhancing model fit.

4. Results

Performance metrics: Table 1 summarizes the performance metrics for each method, including accuracy, precision, recall and F1-score, with standard deviation reflecting variability across cross-validation folds.

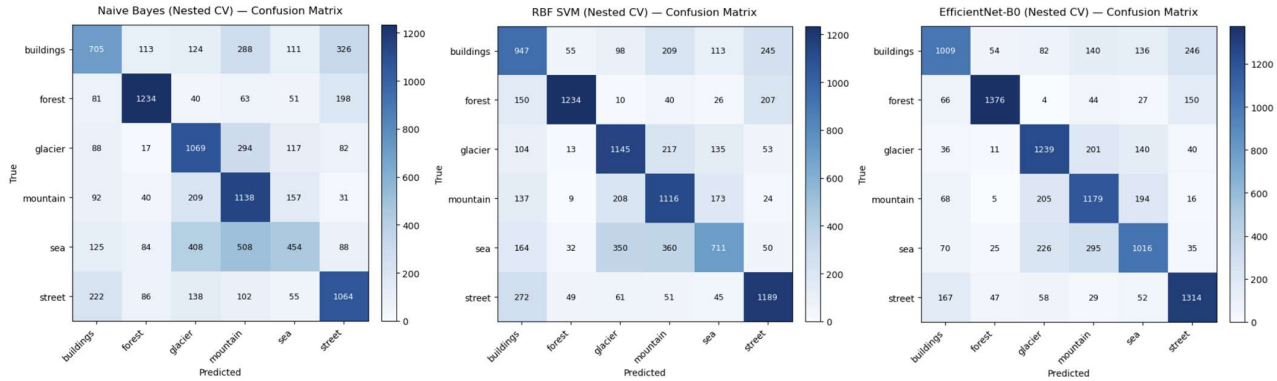
Table 1: Performance Metrics of Proposed Methods (tested on clean images)

Method	Accuracy	Precision	Recall	F1 Score
Naive Bayes	0.5663 ± 0.0144	0.5685 ± 0.0135	0.5663 ± 0.0152	0.5566 ± 0.0148
RBF-SVM	0.6341 ± 0.0165	0.6436 ± 0.0179	0.6341 ± 0.0174	0.6336 ± 0.0171
EfficientNet	0.7132 ± 0.0090	0.7198 ± 0.0087	0.7132 ± 0.0090	0.7133 ± 0.0091

Robustness metrics: The table below shows the performance metrics for each method (tested on altered images)

Alteration Type	NB's Accuracy	NB's F1	SVM's Accuracy	SVM's F1	EffNet's Accuracy	EffNet's F1
Occlusion	0.4676 ± 0.0063	0.4609 ± 0.0054	0.5363 ± 0.0146	0.5399 ± 0.0142	0.6332 ± 0.0130	0.6346 ± 0.0134
Lighting	0.4898 ± 0.0130	0.4783 ± 0.0145	0.5904 ± 0.0130	0.5897 ± 0.0131	0.6799 ± 0.0119	0.6798 ± 0.0124
View	0.4415 ± 0.0092	0.4464 ± 0.0098	0.5401 ± 0.0135	0.5458 ± 0.0133	0.6758 ± 0.0087	0.6756 ± 0.0078

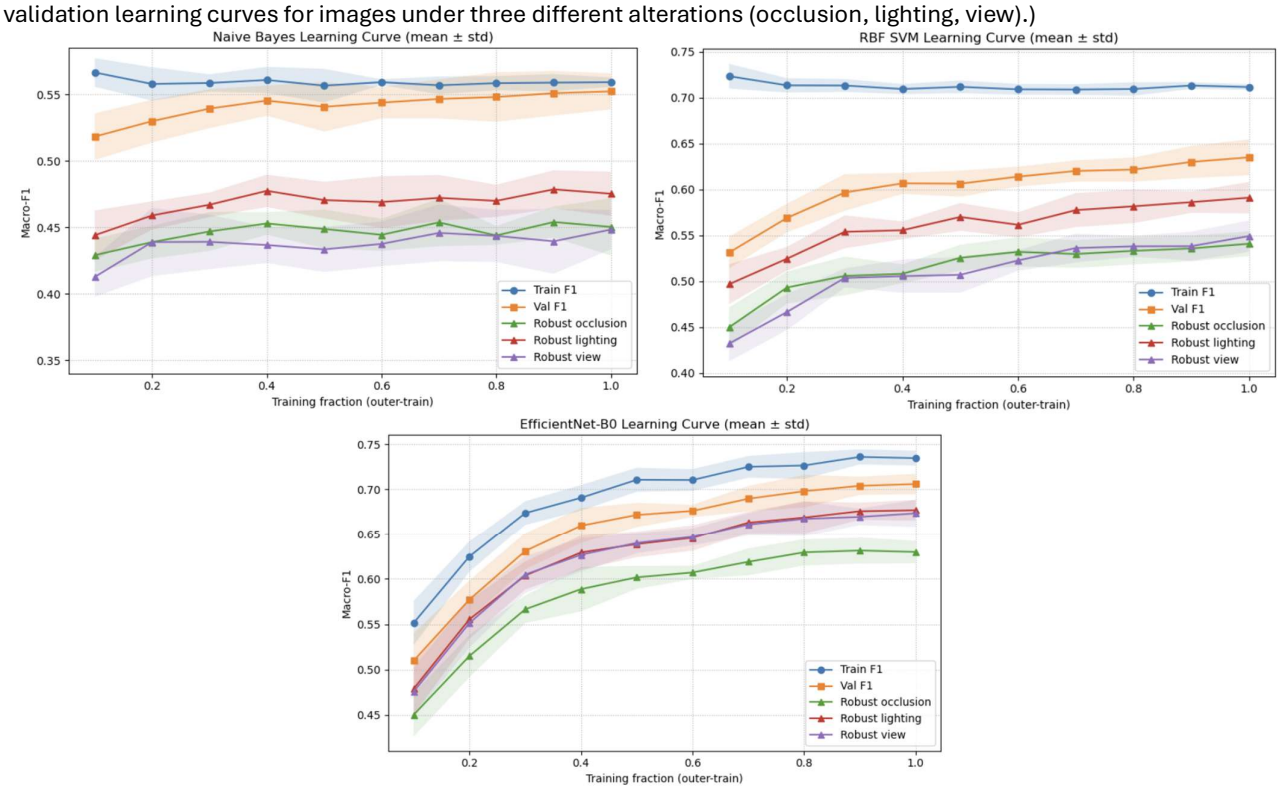
Confusion Matrices: The below graphs show the confusion matrices for the proposed three methods (tested on clean images)



Hyperparameters: The best hyperparameter for proposed three methods.

- Naive Bayes: PCA = 32, var_smoothing = 1e-3
- RBF-SVM: PCA = 32, C = 1
- Decision Trees: LR = 5e-3, WD = 0.3

Learning Curves: The below graphs show the train and validation learning curves for proposed three methods, and validation learning curves for images under three different alterations (occlusion, lighting, view).)



Overall Ranking: EfficientNet > RBF-SVM > Naïve Bayes. The F1 score gap between EfficientNet and RBF-SVM is ≈0.080 (=0.7133-0.6336), which is ≈8.76× EfficientNet's standard deviation and ≈6.43× RBF-SVM's standard deviation, so the separation is decisive. Similarly, RBF-SVM is also superior to Naïve Bayes by ≈0.077 (=0.6336-0.5566), which is ≈5× each model's standard deviation.

Robustness to Image Alteration (Altered vs Clean):

- Naïve Bayes: F1 drops 0.096≈0.5566-0.4609 (occlusion), 0.078≈0.5566-0.4783 (Lighting), 0.110≈0.5566-0.4464 (view)
- RBF-SVM: F1 drops 0.094≈0.6336-0.5399 (occlusion), 0.044≈0.6336-0.5897 (Lighting), 0.088≈0.6336-0.5458 (view)
- EfficientNet: F1 drops 0.079≈0.7133-0.6346 (occlusion), 0.034≈0.7133-0.6798 (Lighting), 0.038≈0.7133-0.6756 (view)

Lighting is the least impactful; View and occlusion changes are detrimental. EfficientNet degrades the least across all three distortions.

Class-wise Behavior: Forest is the easiest (Accuracy: NB 0.74≈1234/1667, RBF-SVM 0.74≈1234/1667, EfficientNet 0.83≈1376/1667); Sea is the hardest for NB (0.27) and RBF-SVM (0.43), and it's the second hardest for EfficientNet (0.61); Building is the hardest for EfficientNet (0.61), and it's the second hardest for NB (0.42) and RBF-SVM (0.57).

Learning-Curves (description focus on train F1 and valuation F1):

- Naïve Bayes: flat curves; small train-val gap. High bias. More data yields marginal gains. The curve stabilizes around 0.56, suggesting a performance ceiling.
- RBF-SVM: steady valuation gains with data; more data yields smaller gap from train F1 to val F1, indicating excellent generalization as dataset size increases.
- EfficientNet: more data yields steady improvement of train F1 and val F1; smaller gap than RBF-SVM. The curve seems plateaus at 0.71, indicating limited performance as epoch fixed at 10 (due to computation limit).

5. Discussion

Performance differences: EfficientNet outperforms the classical methods (F1 = 0.7133 vs RBF-SVM's F1 = 0.6336 vs NB's F1 = 0.5566). Its advantage persists under shifts (occlusion/lighting/view F1 = 0.6346/0.6798/0.7133 vs RBF-SVM 0.5399/0.5897/0.5458 vs NB 0.4609/0.4783/0.4464).

Analysis and Insights:

- Naïve Bayes: Pixels are strongly correlated within textures and edges. The conditional-independence assumption is violated. NB cannot model complex joint patterns, so it underfits. Learning curves are flat and the train-val gap is small, showing high bias. Confusion concentrated on visually similar pairs (glacier/mountain, building/street).
- RBF-SVM: The RBF kernel adds nonlinearity and improves class separation, hence higher precision/recall than NB. Still, flattening with PCA discards spatial layout. Under occlusion/view changes, decision boundaries rely on global texture, so robustness gains are modest. Learning curves rise with data and show moderate bias and variance.
- EfficientNet: Pretrained mobile inverted bottleneck convolution blocks and squeeze-and-excitation encode edges, textures, and part-whole relations that match scene cues. Strong augmentation and weight decay limit overfitting. This yields higher F1 score on confusing classes (e.g., sea/glacier, buildings/streets) and the smallest robustness drop because spatial features and invariances survive occlusion, lighting, and view changes.

Advantages and Disadvantages:

Naïve Byes:

- Pros: computationally efficient; stable across folds; simple to analyse. It's ideal as a baseline.
- Cons: high bias; weak robustness; limited performance even with more data due to independence assumptions.

RBF-SVM:

- Pros: handles high-dimensional, non-linear data effectively. It's a strong classical baseline.
- Cons: computationally intensive and sensitive to parameter tuning; ignores spatial structure.

EfficientNet:

- Pros: best F1-score; more data improves performance; robustness to alteration as it integrates global cues.
- Cons: highest training cost (needs a GPU); pre-trained model; performance depends on LR/WD/Epoch tuning.

Alternative Approaches Considered: Larger CNNs or transformers will likely have higher performance ceilings but considerably higher computation cost and tuning time for 10×3 nested cross validation and will require more training data from scratch.

6. Conclusion

This project evaluated three machine learning algorithms – Naïve Bayes, RBF-SVM, and EfficientNet – for classifying Intel Image Classification dataset. EfficientNet offers the highest and most stable performance (F1 = 0.7133) with smaller fold variance (± 0.0091) than Naïve Bayes (0.5566 ± 0.0148) and RBF-SVM (0.6336 ± 0.0171) classical baselines, because it preserves spatial patterns and adapts invariances that match real-world shifts. RBF-SVM is competitive second but lose robustness because PCA flattens spatial structure. Naïve Bayes underfits due to conditional-independence assumption. Overall, this work demonstrates that effective classification hinges on thoughtful algorithm selection, feature enhancement, and hyperparameter tuning.

Reference

Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). *A training algorithm for optimal margin classifiers*. 144–152.
Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks*. 6105–6114.